

LIFE IS LEARNED — HANDOFF 006B

Private published-books repo + Cloudflare Worker catalog, preview covers and downloads

Authoritative replacement for Handoff 006 public-distribution architecture. 2026-10-10.

Product: Life Is Learned (SwiftUI), owner [Patchagray](#) on GitHub. **Implementation baseline:** [Patchagray/LifelsLearned](#), [feature/handoff-005-library-platform](#), observed at [19c73333ac3d0fdf26541de2096c2452c0c01c33](#) on 2026-10-10. Confirm latest branch head before making changes. **Proposed branch:** [feature/handoff-006-private-cloudflare-explore](#).

1. Intent and non-negotiable release gate

Build a native **Your Library | Explore** switch inside the existing home Library section. Explore retrieves a metadata-only catalog and separately cached cover thumbnails from a Cloudflare Worker. A user taps Download/Restore; Worker reads the matching approved release from a **private GitHub published-books repo**. The app validates bytes, SHA-256, stable book ID, revision and complete package, then atomically installs. Existing downloaded books, including studio MP3s, remain readable offline. Keep the + menu for **Scan a Book** and **Import File** only.

Critical owner instruction: create an EMPTY PRIVATE publication repo, but DO NOT move/publish ANY book in this handoff. The owner will later instruct Codex which books are approved. A book is eligible only if **(a)** the owner explicitly approved that exact book and revision for distribution, **(b) every idea** contains a complete, valid **ElevenLabs** narration bundle with both approved Guide and Storyteller roles, all pages, questions, feedback branches and completion scores, and **(c)** the exact final package passes existing authoring and audio gates. No books using device/local TTS as their only narration, or with partial/corrupt/stale premium bundles, enter the published repo. Publication approval is independent of audio test success; passing tests is not permission to publish.

Covers are a separate artifact: a **cover preview may appear for a Coming Soon title only after separate explicit approval to publish that cover image**. This does not approve its book package or change its status to downloadable. Unapproved covers must stay private in the production repo. A book package is never added just to expose its cover.

No auto-migration from the original private production repo, past AB001 book batches, device imports or existing manual collections. Do not infer approval from "produced," "uploaded," a prior ZIP, or a book title. No migration of the four books the owner mentioned until a separate explicit instruction, identity checks and audio gate pass.

2. Private repository setup — do this early

Create [Patchagray/LifelsLearned-Published](#) as **PRIVATE**, distinct from [Patchagray/LifelsLearned](#) (app/source) and the existing private production/authoring repository. If that exact name is taken, stop and ask the owner for an alternative; never reuse or expose a public repo. Repo creation itself is approved now; **book publication is not**. Use the existing authorized GitHub account ([gh](#) on the user's Mac/Codex runtime if authenticated); do not request or paste PATs into chat.

The ZIP contains [PrivateRepoScaffold/](#) and [Bootstrap/bootstrap_private_repo.sh](#), which establish a safe zero-book initial state. Copy [Catalog/Remote-Catalog-001.json](#) from the app/source repo verbatim into the new

repo at `catalog/Remote-Catalog-001.json`; its 50 planned Catalog 001 entries must remain truthful. Initialize the two empty allowlists, create the repository, confirm `isPrivate == true`, then commit/push **only** initial scaffold + planned metadata. No MP3s, books, ZIPs or generated release assets. Keep required visibility private and verify it again before every subsequent release upload.

Suggested private repo layout:

```
LifeIsLearned-Published/      [PRIVATE]
  README.md
  .gitignore
  catalog/
    Remote-Catalog-001.json    # exact 005-compatible catalog; 50 planned at bootstrap
    approved-packages.json    # {schemaVersion:1,entries:[]} at bootstrap
    approved-covers.json      # {schemaVersion:1,entries:[]} at bootstrap
  covers/                     # separately approved preview JPEG/PNG only
  approvals/                  # one explicit owner approval record per release
  [GitHub Releases assets]    # later, only approved packaged-audio JSON
```

`approved-packages.json` acts as a **server-side allowlist**; its entries include book ID, collection revision, GitHub **private release asset ID**, SHA-256, exact bytes and approval-record reference. `approved-covers.json` separately maps canonical book IDs to safe file paths/bytes/SHA-256. Do not publish a release just by moving a JSON to `main`: the Worker exposes books only when an approved package allowlist entry, corroborating approval and actual asset are valid. Deploy the Worker in a **closed fail-safe empty catalog state** if either private repo or credentials are not configured.

3. Cloudflare Worker architecture

Deploy a minimal Cloudflare Worker, e.g. `lifeislearned-catalog`, with GitHub App **installation authentication** scoped to the new private repo, **Contents: read-only**, and only the additional metadata permission strictly necessary for release retrieval. Use a GitHub App with least privilege, JWT -> short-lived installation token (~1 hour); cache token safely in worker memory or a suitable secure store, renew as needed. Never embed the GitHub App private key or token in app code, Git, catalog, URLs, screenshots, logs or build settings. Store credentials with Cloudflare **Worker Secrets**, not `[vars]`. A fine-grained read-only PAT may be a **temporary development fallback** if App setup is blocked, but only as Worker secret scoped to this repo; document retirement and token-rotation plan. No GitHub credentials ever go to the iOS app.

Public-facing endpoints (versioned; no raw GitHub host/paths/asset IDs in client metadata):

Request • Result • Gate

`GET /v1/catalog • DiscoveryCatalog schemaVersion 1 / catalogID catalog-001 / catalogRevision 1 • read-only; overlay explicitly allowlisted covers/packages on canonical 50 planned entries`

`GET /v1/covers/{bookID} • separately approved JPEG/PNG, <=512 KiB • approved cover preview entry for exact book ID + checksum/size`

`GET /v1/books/{bookID}/download • streamed exact approved JSON package bytes • approved package allowlist + owner approval record + valid private asset`

`GET /healthz • non-sensitive up/down status • no secrets, repo IDs or inventory`

The Worker must **not** expose arbitrary file paths, raw GitHub API proxying, list-all releases, private directory contents or a "download by asset ID" route. Reject unknown IDs/methods; restrict IDs to the frozen Catalog 001 identity. Pin GitHub owner/repo in trusted Worker configuration. Fetch approved release assets via GitHub's authenticated **Get release asset** endpoint; follow GitHub's 200 or 302 behavior **server-side**. Never forward GitHub Authorization headers to a redirect host, and never return the GitHub redirect Location to the app. Only stream the verified allowlisted asset; do not buffer 64-MiB book JSON in Worker memory. Respect upstream

status codes, content length and redirect limits. Set appropriate content type and [Content-Disposition: attachment](#); omit sensitive headers. Only send claimed package byte count and SHA-256 after publication metadata has been validated; the app remains the final verifier.

Ranges/resume: The 005 [URLSessionDownloadTask](#) may issue Range/resume requests. Proxy [Range](#), [If-Range](#) and [206/Content-Range](#) accurately if upstream supports them; never synthesize invalid partial responses. If unsupported, document and test safe clean restart from byte zero. Existing cancel/retry behavior must remain reliable. Worker routes must support sensible timeouts and bounded requests, and apply rate limiting/bot mitigation appropriate to a prototype without implying true client identity proof.

Security boundary (important): GitHub repository contents remain private from direct anonymous GitHub access, but without App Attest or user authentication, **anyone who learns the Cloudflare endpoint can request its published book downloads**. Do not claim "only our app can access these books," DRM, or extraction prevention. App Attest is explicitly deferred to a later commercial/release milestone. Publish only content the owner is comfortable serving to holders of Worker URLs. Optional rate limits discourage abuse but do not authenticate the caller.

4. Exact current iOS catalog contract: do not invent a new package format

The 005 source already has:

- [LifelsLearned/Views/LibraryView.swift](#) with [Your library](#), current grid, toolbar [+](#), and [Browse Library](#) in the confirmation dialog.
- [LifelsLearned/Views/DiscoveryView.swift](#), [DiscoveryStore](#), [DiscoveryService](#), [BookDownloadManager](#) and [PackageDownloadService](#).
- [LifelsLearned/Models/DiscoveryCatalog.swift](#), [RemoteAsset](#), [RemotePackage](#), [DiscoveryBook](#), stable Catalog 001 identity and SHA-256/byte-bound checks.
- [LifelsLearned/Services/RemoteConfiguration.swift](#), with [DiscoveryCatalogURL/BookRequestAPIURL](#) mapped from reviewer-controlled Xcode build settings.
- [Library-v3](#) preservation of idea cards, favorites, progress, History and offload/reinstall.

The Worker response must preserve the app's established **schemaVersion 1** shape. [DiscoveryBook](#) fields are [id](#), [title](#), [authors](#), [primaryShelfID](#), [secondaryShelfIDs](#), [tags](#), [isbn13](#), [availability](#) ([available](#)|[planned](#)|[unavailable](#)), optional [thumbnail:{url,sha256,bytes}](#), optional [package:{collectionRevision,url,sha256,bytes}](#). [available](#) requires the package object; [planned](#) never offers Download. URLs point at Worker HTTPS endpoints, **not private GitHub**. All stable book IDs and shelf assignments match [Catalog/Catalog-001.json](#). Keep the catalog under 2 MiB, thumbnails ≤ 512 KiB and encoded JSON package ≤ 64 MiB; use the 005 validator and client checks for deeper resource/format limits. Cache metadata and thumbnails in app for fast scrolling; no full package download while browsing.

Recommended server composition: start from the checked-in canonical 50-book planned JSON; overlay [approved-covers.json](#) into [thumbnail](#), and overlay [approved-packages.json](#) into [package](#) + [availability:"available"](#) **only if matching verified owner approval exists**. All other entries remain planned; no inference from presence of an unapproved file in GitHub. Return no internal asset IDs, private repo references or approval file paths.

5. In-place Explore UX — same decision as original 006

At the existing library-section heading, add an accessible segmented **Your Library | Explore** switch; both are equal states within the same home scroll region. Your Library remains the installed/offloaded learner shelf/grid. Explore swaps that section only to remote shelf/catalog browsing. Do **not** create a bottom tab, modal Explore screen or replace the home hero. Search, labels, item count, shelf choices and empty states follow active mode; provide loading/offline/cached/retry states. Show available cover previews even for **Coming Soon** books if an approved thumbnail exists. Display placeholder artwork if absent or offline. Show **Download, Update, Installed, Coming Soon, Download Again, or Request** as appropriate. Book details can be pushed within existing NavigationStack.

The top-right + confirmation dialog contains **only Scan a Book, Import File, Cancel**. Remove **Browse Library**; do not rename it to **Browse Catalog** there. Scanner matches and remote-book restore should select Explore and focus the matching ID, not navigate to the old Browse Library view. Manual-import offloaded books still require manual file re-import. Preserve continued learning, global Ideas, Settings and active narration, and preserve learner data across reimport/update/offload. Present Explore even with no installed books; fix current `if !library.books.isEmpty { librarySection }` gating.

Reuse the present 005 DiscoveryService/BookDownloadManager, remote catalog validation, staging, cancellation/resume, SHA-256 and atomic install pipeline. Refactor DiscoveryView into a reusable in-place section rather than fork its state/data logic. Debug fixture routing remains distinct from real deployment; no production endpoint assumptions in lesson packages.

6. Premium narration/owner approval — two independent gates

A. Automatic technical gate (must PASS for all ideas)

Run `Tools/validate_package.py PACKAGE.json --authoring-gate` and `Tools/validate_package.py PACKAGE.json --audio-gate` using the app repository's existing scripts/FFmpeg setup, recording machine-readable reports and process exit codes. Verify every lesson has a **narration** bundle, every expected segment is present with correct role, exact script hash, decodable mono MP3 and valid duration, including **all feedback choices and all three completion scores**. Verify audio `provenance.provider == "elevenlabs"` for every lesson with approved production Guide/Storyteller voice identifiers and accepted script/production metadata; if provenance is absent/mismatched, block. Confirm that **no idea would invoke local TTS at playback preflight**. Empty, partial, corrupt, stale or synthetic-tone fixture audio is a hard publication failure even though the normal reader can fall back. Validate full 300-second packaged-audio worst branch (the existing `--audio-gate` rule) and structural and image budgets. Deep Dive/Deeper content and its optional narration remain **out of this gate** until separately approved; absence of deeper narration must not block otherwise approved core book release.

The regular 005 runtime continues supporting fallback for manually imported books. **Do not change that behavior**. This is a stricter **distribution/publishing policy**, not a breaking schema requirement for every imported book.

B. Owner editorial/release approval (must be explicit)

A human must approve the **exact book ID, revision, final file SHA-256, audio QA report and voices**. Approval is bound to the final package hash in a private approval record (schema documented in [PrivateRepoScaffold/approvals/README.md](#)). Passing automated tests is not equivalent to authorization. Only after approval, Codex can upload that exact file as a private GitHub Release asset and update its allowlist. Never create inferred approvals. Any byte change, metadata-only package change, voice replacement or changed script requires a new SHA-256 and fresh release approval (even when collection revision rather than idea revision changes). Releases may remain retained for rollback; only latest approved allowlist revision becomes discoverable. Prevent silent rollbacks or downgrades.

Publication transaction order: verify private repo -> validate package/complete premium audio -> obtain owner approval for hash -> upload immutable versioned GitHub Release asset -> verify uploaded bytes/asset ID -> add private approval record -> update [approved-packages.json](#) **last** -> Worker serves new catalog. Publish cover preview only after separate owner preview approval, independently of book completion. A failed transaction leaves book status planned or prior revision available, with no broken download exposed.

7. Repo and security controls

- GitHub App installed on **published-books repo only**, not on production/source repositories; `contents:read`, minimal metadata; Worker secrets for `GITHUB_APP_ID`, `GITHUB_INSTALLATION_ID`, `GITHUB_APP_PRIVATE_KEY`. App public key/secrets are never in Git.
- Private repo creation must verify `gh repo view Patchagray/LifelsLearned-Published --json isPrivate` returns true. Abort if repo is public or if authenticated account lacks access.
- No raw ElevenLabs masters, prompt sheets, transcripts, authoring research, audio-generation configurations, unpublished books, test tones, personal details, PATs, Cloudflare API tokens or `.dev.vars` in published repo.
- Production repo remains private. Worker does not provide arbitrary GitHub path, branch or asset-ID API. Cache response metadata separately from book binaries; documented invalidation on manifest revisions.
- Failure cases: GitHub 401/403/404/429, Worker 5xx, missing approval, asset deleted/mismatch, malformed private manifest, incorrect preview hash, network loss, revoked GitHub App, stale cache. Fail closed; do not relabel planned books as available or corrupt installed learner state.
- Deploy staging Worker and test first. Reviewer controls production Worker deployment/secrets and any public access. App Attest and authenticated authorization deferred deliberately, not quietly represented as delivered.

8. Verification / tests required to close 006B

1. **Private repo creation proof:** correct owner/name, `private:true`, zero uploaded book packages, both allowlists empty, canonical catalog 50 planned, no production leakage. If not authorized to create, provide exact blocker; **never claim created**.
2. **Publication refusal:** reject legacy/local-TTS-only book; reject one missing/stale/corrupt MP3; reject fake "ElevenLabs" provenance without validated audio; reject synthetic fixture; reject absent owner approval, wrong package SHA/revision, and partially approved 10-idea collection. Prove approval alone cannot bypass audio gate, and audio gate alone cannot bypass owner approval.
3. **Catalog preview:** 50 planned book metadata entries served; approved cover appears on planned title; unapproved cover is hidden; cover failure uses placeholder, no whole-book transfer; preview cannot grant Download.

4. **Private-download proxy:** unauthenticated direct GitHub access denied; Worker fetches approved private asset; app receives only Worker URL; tampering/size/hash/ID mismatch rejected; no leaked tokens/redirect Location. Verify Range/206 or clean restart; cancellation/retry and offline cached state.
5. **iPhone/iPad UI:** Library/Explore toggle in-place even with empty library, labels/count/search/shelves/status, preserved continue-learning, scanner/remote restore focuses Explore, plus only Scan/Import, VoiceOver/Dynamic Type, existing reader/cards/covers unaffected.
6. **Storage:** download -> SHA verify -> atomic import -> offline studio playback -> offload -> History/card/progress preserved -> redownload and correct premium audio. Negative install failure retains original state; no app-wide fallback introduced.
7. **Operational evidence:** commands, network mock/real smoke, staging endpoint, screenshots, signed build as applicable, package approval test results, git SHA, repo URL and `private:true` check, Worker secrets existence only (never secret values), unresolved physical and App Attest future checks.

9. Delivery, sequencing and stop points

006B.1: inspect current 005 branch, rebase safely if owner work progressed. **006B.2:** create private **empty** published repo + planned catalog/allowlists; stop short of publishing books. **006B.3:** build and test Cloudflare Worker in staging with mocked GitHub responses; configure scoped GitHub App/Worker secrets via owner-controlled account flow. **006B.4:** wire native Explore section, cover fetch/cache, scanner/restore routing and + menu. **006B.5:** implement strict future publication preflight/approval guard and fail-closed overlay; run regression. **006B.6:** optional zero-release real staging smoke and owner-operated physical checks. **DO NOT publish any book** in this handoff. **006B.7:** deliver `HANDOFF_006B_IMPLEMENTATION_REPORT.md`, test artifacts and exact repo/Worker status, and wait for the owner's separate instruction naming approved book releases.

10. Explicit non-goals

No book generation, no ElevenLabs generation, no auto-transfer of previously produced books, no production repo visibility changes, no public distribution repo, no user accounts/payments, no App Attest yet, no DRM claims, no narration architecture rewrite, no seventh core lesson page, no Deep Dive audio gate, no new bottom nav. **Only the empty private repo and Worker/catalog infrastructure are authorized for setup now.**

Technical references

- GitHub private release assets: <https://docs.github.com/en/rest/releases/assets> (Contents read permission; 200 stream or 302).
- GitHub App installation tokens: <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-an-installation-access-token-for-a-github-app> (short-lived, scoped).
- Cloudflare secrets: <https://developers.cloudflare.com/workers/configuration/secrets/> (secrets not vars).
- Cloudflare Worker limits: <https://developers.cloudflare.com/workers/platform/limits/> (streaming recommended; avoid large in-memory buffering).